Breaking Isolation: Last-level Cache Side-Channel Attacks on AWS Nitro Enclaves

Monder Rammouz
TU Munich
Germany

Quoc-Do Le
Huawei Munich Research Center
Germany

Abstract

Trusted Execution Environments (TEEs) safeguard the confidentiality and integrity of application code and data by creating isolated, secure memory regions known as *enclaves*, even in the presence of privileged adversaries with root or physical access. While TEEs such as Intel SGX, Intel TDX, and AMD SEV-SNP have been shown to be vulnerable to side-channel attacks, AWS Nitro Enclaves adopt a different design that avoids shared cores and memory regions. According to the AWS Whitepaper, by utilizing exclusive CPU cores and enforcing memory isolation through the Nitro Hypervisor, Nitro Enclaves prevent shared cache usage with other instances. The design treats cache-based side-channel attacks as impractical and outside the expected threat model.

This paper presents an empirical study showing that Last-Level Cache (LLC)-based side-channel attacks can, in fact, be mounted against AWS Nitro Enclaves in real production environments. While our attack builds on established cache side-channel techniques, we demonstrate that the parent AWS EC2 VM and its enclave share the LLC, enabling practical observation of enclave activity.

CCS Concepts: • **Security and privacy** → **Operating systems security**; **Side-channel analysis and countermeasures**; • **Computer systems organization** → *Cloud computing*.

ACM Reference Format:
Monder Rammouz and Quoc-Do Le. 2026. Breaking Isolation: Last-level Cache Side-Channel Attacks on AWS Nitro Enclaves. In *9th Workshop on System Software for Trusted Execution (SysTEX '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3805690.3805729

1 Introduction

Cloud computing lets users focus on building applications while providers handle infrastructure, but this reliance also raises serious security and privacy concerns. To address these challenges, hardware-based TEEs like Intel SGX [5, 23], Intel TDX [3], ARM CCA [2], and AMD SEV-SNP [38] offer a practical solution to secure workloads [11, 18, 19, 29, 30] in untrusted cloud environments [24, 39]. These TEEs ensure strong guarantees of integrity and confidentiality, even when the underlying software stack-such as the operating system or hypervisor-is compromised.

Despite their promise, TEEs are vulnerable to microarchitectural threats, particularly side-channel attacks [6, 9, 10, 12, 27, 33, 37, 41, 44]. In contrast, AWS Nitro Enclaves present a more robust alternative. By leveraging dedicated hardware and isolated memory, Nitro Enclaves offer enhanced protection against side-channel exploits. Unlike processor-bound TEEs [39] such as Intel SGX, Intel TDX, AMD SEV-SNP, or ARM CCA, AWS Nitro Enclaves rely on an external module [1], which does not execute arbitrary code and can be independently replaced, making it inherently more resistant to such attacks[7, 26].

According to AWS documentation [4], the Nitro hypervisor enforces several security constraints: (1) CPU core #0 and its threads are excluded from enclave assignment, (2) enclaves must utilize full cores, and (3) all cores assigned to an enclave must reside within the same NUMA node. However, a key vulnerability remains, Nitro Enclaves may share the LLC with their parent EC2 instance or other enclaves. Recent studies [48] have demonstrated that LLC side-channel attacks are feasible in production environments, raising the question: *Can the shared LLC be exploited to launch side-channel attacks against AWS Nitro Enclaves?*

Although LLC-based attacks are inherently powerful, their execution in modern public cloud environments is considerably more challenging [48], and even more so within Nitro Enclaves. In cloud settings, hardware resources are shared among multiple tenants to maximize compute density, which saturates the LLC with background activity. Figure 5 shows the distribution of cycles between background accesses across 2322 randomly selected LLC sets on an EC2 instance (measured at different times of day) compared to our local quiescent environment. On EC2, each LLC set experiences an average of 6.26 accesses per million cycles—more than 1300× the rate observed in our noise-free local environment (0.0048 accesses per million cycles). Noise introduces several practical obstacles for attackers. First, eviction set construction becomes unreliable, as false evictions lead pruning algorithms to discard valid congruent addresses or mistakenly accept incongruent ones. Second, timing pattern extraction is hindered, since victim-specific traces are obscured by co-tenant

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN /2026/04
https://doi.org/10.1145/3805690.3805729

accesses. In addition, priming phases are more prone to false negatives, making it difficult to recover closely spaced accesses within the required time window. For an attack to succeed, the attacker must achieve temporal overlap between the enclave's short-lived workload and monitoring of the correct eviction set. This synchronization problem is exacerbated by the large number of LLC sets, where monitoring may coincide with unrelated activity, reducing hit rates. In AWS Nitro enclaves, enclave cores are pinned and run without interruption rather than being time-sliced with the parent VM, which makes synchronizing probes with victim activity even harder. Amazon's EC2 security whitepaper [4] explicitly concludes that such attacks are not considered viable threats.

This paper challenges that assumption. Building on prior research [48], we present a complete, end-to-end demonstration of LLC side-channe attacks targeting applications within AWS Nitro Enclaves, proving that, despite the hurdles, these attacks are indeed feasible in modern cloud environments. Our approach accelerates eviction set generation through filtering-based pruning, enables high-resolution monitoring of victim memory access, and applies signal processing to swiftly pinpoint target cache sets. Additionally, we develop noise-resilient heuristics for analyzing cache access traces and extracting secrets under noisy cloud conditions. To summarize, we make the following main contributions.

- We uncover a side-channel vulnerability in AWS Nitro Enclaves arising from the unexpected sharing of the LLC between the parent EC2 VM and the Nitro Enclave.
- We present a comprehensive end-to-end demonstration of cache side-channel attacks targeting naive applications in AWS Nitro Enclaves, illustrating their feasibility in production environments despite substantial obstacles. This vulnerability has been responsibly disclosed to AWS.
- We adapt this attack to additional enclave technologies, including Huawei Qingtian Enclave and Intel SGX, with disclosures provided to the respective vendors.
- We develop noise-resilient, training-free heuristics that detect single secret-dependent memory accesses in implementations that are constant-time at the architectural level (i.e., no input-dependent runtime differences), yet still exhibit microarchitectural leakage.

2 Background

2.1 AWS Nitro Enclaves

AWS Nitro Enclaves [1] are a type of TEE that leverages the hypervisor to provide isolation by creating the TEE as a separate virtual machine (VM), referred to as the "enclave VM," alongside the "parent VM" (see Figure 1). While other TEEs, such as Intel TDX and AMD SEV-SNP, are vulnerable to cache side-channel attacks (see §2.2), AWS Nitro Enclaves

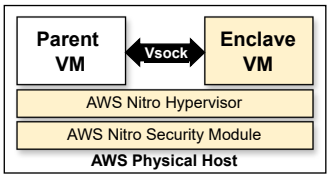

Figure 1. AWS Nitro Enclave Overview.

address this issue by preventing side-channel attacks through measures like never sharing Simultaneous Multi-Threading (SMT)-enabled CPUs between instances, with the same protections applying between the host instance and the enclave, as detailed in the AWS whitepaper [4]. Nitro Enclaves also utilize specialized hardware to provide signatures for remote attestation. All these features are enabled via a kernel module loaded into a Nitro Enclave-compatible EC2 instance. The Nitro Hypervisor is designed to be minimal and relies on Nitro Cards, hardware devices that handle system control and virtualized I/O independently of the CPU. Both the enclave VM and parent VM are connected through a VSock interface.

2.2 Cache Side-Chanel Attacks

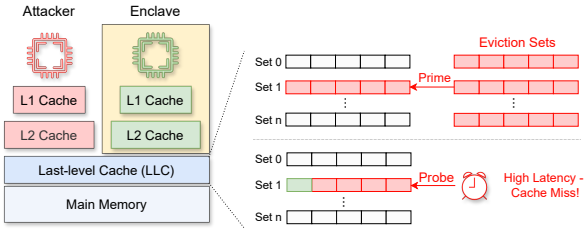

Figure 2. LLC Side-Channel Attack Overview.

Although initially appealing for cloud security, the use of TEEs is vulnerable to cache side-channel attacks [6, 9, 10, 12, 27, 33, 37, 41, 44]. Cache side-channels are a relevant threat in multi-tenant cloud environments and web application backends, where hardware is extensively shared between mutually distrusting parties. They allow an attacker to bypass virtual isolation by observing subtle differences in hardware usage. A malicious tenant co-located on the same physical host can exploit shared microarchitectural infrastructure (e.g. CPU caches, TLBs, branch predictors) to infer another tenant's computation. For instance, cache side-channel attacks can exploit timing differences between cache hits and misses in shared processor caches to infer accesses to sensitive data, such as cryptographic keys, without direct access to victim processes [22]. Key techniques include Prime+Probe (filling and probing cache sets for victim-induced evictions) [20, 28],

Flush+Reload (flushing shared lines and timing reloads) [45], and variants like Evict+Reload [14] or Flush+Flush [13]. Earlier work [15, 16] demonstrated LLC Prime+Probe attacks on AWS EC2 in 2015, but their methods relied on inclusive caches, long-lived processes, and huge pages features that are largely incompatible with today's cloud infrastructure. Consequently, cloud providers have dismissed LLC Prime+Probe attacks as impractical. Recent research [48] demonstrates that LLC side-channel attacks are feasible in cloud production environments, effectively bypassing cloud-induced noise through techniques such as L2 filtering, parallel probing, training-based target set detection and secret extraction, enabling the recovery of over 80% of ECDSA nonce bits within seconds on platforms like Google Cloud. Building on this work, we demonstrate that LLC side-channel attacks (see Figure 2) remain viable even when enclave technologies like AWS Nitro Enclaves (see §2.1) are employed.

3 Threat Model

AWS Nitro Enclaves operate under a threat model that considers the parent EC2 instance, including its users, processes, and administrators, as potentially untrusted [1]. To protect sensitive data and code, Nitro Enclaves use Nitro Hypervisor to enforce strict CPU and memory isolation, blocking access from the parent instance, even for root users. They also disable external networking, persistent storage, and interactive access (e.g., SSH) to reduce the attack surface. The integrity and authenticity of the enclave are verified through cryptographic attestation, which integrates with services like AWS KMS to ensure that only authorized enclaves can handle sensitive operations. Our work adopts this same threat model, assuming that an attacker has full control over the parent instance and attempts to conduct LLC side-channel attacks. In addition, we assume the attacker knows which open-sourced cryptographic implementation is ran by the enclave workload. The attacker can monitor for multiple widely used implementations via their proper access timing patterns to specific address offsets.

4 Nitro Enclave shares LLC with Parent VM

According to the AWS white paper [4], AWS Nitro Enclaves achieve isolation by allocating entire CPU cores and extracting memory pages from the parent virtual machine using the kernel-supported "hot-unplug" feature (see §2.1). These resources are then repurposed by the Nitro Hypervisor to launch a separate, fully secured virtual machine environment dedicated to running the Nitro Enclave image. Our investigation of AWS Nitro Enclaves in a production setting revealed that the LLC microarchitectural resources are shared among the parent VM, enclave VMs, and all co-resident tenants (i.e., other EC2 instances) on the same non-uniform memory access (NUMA) node. As a result, an adversary conforming to our threat model need not share the same core or memory resources with the target enclave; control over the parent VM or a co-located EC2 instance on the same physical host is sufficient to mount LLC-based side-channel attacks [48]. Moreover, such an adversary can monitor enclave workload invocations from a compromised untrusted parent VM. Co-resident tenants within the NUMA node similarly share the LLC, thereby permitting malicious EC2 instances to persistently surveil enclave memory access patterns. The attacker can trigger the workload repeatedly, launch the attack with each invocation, or continuously scan. Frequency-domain techniques like the Power Spectral Density (PSD) are robust here because they detect periodic leakage even when most recorded accesses are not aligned. Finally, only a small leakage is sufficient to mount cryptanalytic attacks, such as key recovery via lattice-based methods.

Cache Contention. In modern Intel servers, LLC is w-way set-associative i.e. organized into sets of w cache lines each, and located via the set index field, partly shared between the physical and virtual address. More than w memory addresses are congruent i.e. map to the same set, resulting in contention: on a cache miss, a line is evicted in accordance with the cache replacement policy (e.g. LRU). A minimal eviction set is the smallest group of addresses guaranteed to evict all lines of an LLC set. Eviction from private cache lines is achieved in modern Intel servers with non-inclusive LLC by exploiting the Snoop Filter (SF), an undocumented set-associative coherence directory. The SF and LLC share the same set mapping, and addresses that collide in the SF also collide in the LLC. As an SF set entry is evicted, the managed private line is invalidated and demoted depending on an internal reuse predictor. LLC eviction sets, which are also SF eviction sets, thus evict lines across all levels of the cache hierarchy.

Confidential Applications. AWS Nitro Enclaves are used to ensure the confidentiality and integrity of confidential applications that contain secrets, such as an ephemeral key used in a cryptographic operation (e.g. signing or encryption), or sensitive data (e.g. credit card numbers, healthcare data). In this paper, we target cryptographic implementations fulfilling the constant time property. In the absence of timing-differences, secret-dependent branching can still, through an instruction fetch or a data read, conditionally access a specific memory address offset, utilizing the cache. The constant time property acts in favor of access pattern identification in the frequency domain and filtering out noise in the yielded traces, as secret-dependent accesses follow the periodicity of the targeted cryptographic implementation. Finally, we observe that containerization preserves timing behavior of the victim program within the enclave.

5 End-to-End Attack

The end-to-end LLC side-channel attack is demonstrated by launching an AWS Nitro Enclave from a parent EC2 instance,

constructing eviction sets on the parent VM, and running a test application inside the enclave that repeatedly processes a secret. Using Prime+Probe combined with signal processing, the attacker scans LLC sets to detect secret-dependent access patterns, monitors the targeted sets to reconstruct the secret, and validates the results against the known data. While this approach follows prior work [48], it introduces new challenges, especially, enclave cores are pinned and run continuously rather than time-sliced with the parent VM, reducing opportunities for synchronization and making alignment with victim activity significantly harder.

5.1 Building Eviction Sets

Attackers use Virtual Addresses (VAs) to build eviction sets that persist across an EC2 instance's lifetime. For 4KB pages, the page offset of a VA and its Physical Address (PA), the least significant 12 bits, matches. When attackers know the secret-dependent offset accessed by the victim program, they target LLC sets mapped to that offset; otherwise, they construct eviction sets for all 64 offsets. Because attackers lack higher-order indexing bits, they only partially control VA set assignment. They gather a candidate pool of potentially colliding VAs and prune it into minimal eviction sets. Pruning algorithms differ in strategy but all test cache-line eviction. In production, noisy accesses increase false positives, so attackers must finish quickly to reduce interference. Prior work [48] exploits the fact that addresses with different L2 set indexes cannot share an LLC set, which reduces incongruent candidates before applying binary-search pruning.

This approach accelerates construction and makes it practical in production. We construct 1,152 eviction sets for a known page offset on an `r6i.2xlarge` instance with a 12-way Ice Lake SP (Intel Xeon Platinum 8375C CPU, 54 MiB LLC). Across 640 measurements, we record an average construction time of 11.06s.

5.2 Monitoring LLC Sets Accesses

In the next step, attackers scan potential LLC sets and identify the actual targets where the victim program produces characteristic access-timing patterns. They can use the Prime+Probe approach to achive the goal. In the Prime phase, they fill all ways of a set with controlled lines from the eviction set. In the Probe phase, they repeatedly re-access those lines and measure latency. A sharp increase indicates an eviction, revealing both the occurrence and timing of a memory access. Parallel probing improves time resolution by overlapping accesses, while the unprivileged instruction *rdtsc* provides precise timestamps by reading the Time Stamp Counter (64-bit model-specific register (MSR)) [47].

Building on prior work [48], we measure the detection rate of accesses to a target set using Prime+Probe with parallel probing latency of 164 cycles (≈ 68 ns at 2.4 GHz), within 500 cycles (≈ 208 ns) of their emission at a fixed interval. In the EC2 production environment, we achieve a detection rate of 97.26% across 10 million accesses emitted with a granularity of 4,850 cycles.

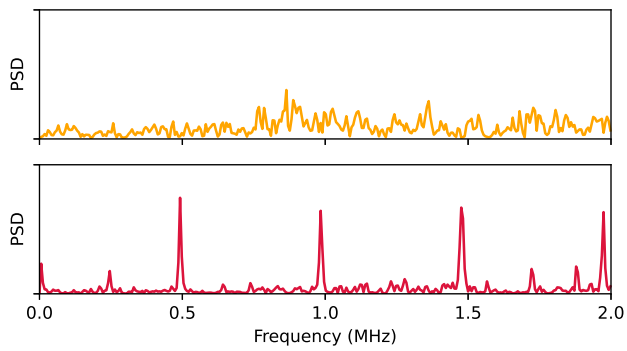

Figure 3. The power spectral density of memory accesses to the target set (red) and to a non-target set (orange) collected on the parent EC2 instance.

5.3 Identifying Enclave Workloads

The attacker iteratively records access traces from each eviction set to identify the target set, the one showing timing patterns characteristic of the confidential workload inside the AWS Nitro enclave. Even constant-time cryptographic implementations can leak secrets when they periodically access memory offsets in a secret-dependent manner.

For example, the ECDSA Montgomery ladder in OpenSSL 1.0.1*e* exhibits secret-dependent control flow [48] (i.e., a flawed ECDSA implementation). Each loop iteration over secret nonce bits performs identical arithmetic, but a branch on the current bit selects different basic blocks on distinct instruction cache lines. As a result, instruction/data-fetch timing in the target LLC set becomes a deterministic function of the secret bits.

In practice, the loop's `if` block is fetched at every iteration boundary, while one branch path triggers an additional mid-iteration fetch of the same cache line, producing a midpoint access pattern [48]. We show that even a single access suffices to enable reliable secret extraction from noisy traces (see §5.4). For each trace, we estimate the PSD using Welch's method. The target set produces peaks at the secret-dependent access frequency (CPU frequency ÷ loop period cycles) and harmonics, yielding a noise-tolerant fingerprint. The attacker detects a target trace when peaks at multiple thresholds fall within a tolerance window around expected frequencies. This approach achieves detection and false-positive rates in production that are comparable to those of a prior SVM classifier trained on 81,345 traces, but requires neither training nor synthetic data. Figure 3 shows PSDs of non-target sets versus the target set, which records secret-dependent accesses at 4,850-cycle iterations, producing peaks at 0.494MHz and harmonics.

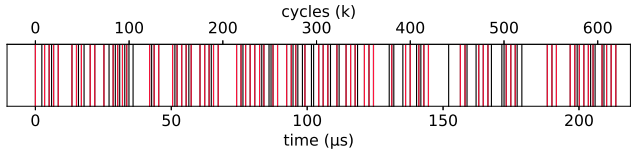

Figure 4. Output of secret-dependent access detection (red), allowing the recovery of a 128-bit key snippet containing four bit errors from the target trace.

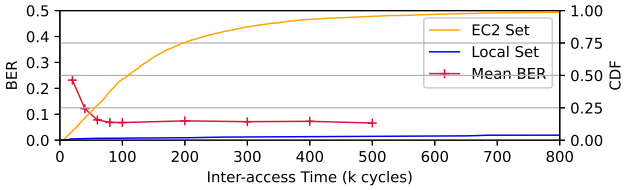

Figure 5. Noise Distribution in EC2 vs. Local LLC Set and Resulting Side-Channel Attack Performance.

The enclave executes the vulnerable loop during 25% of its runtime. We continuously scan eviction sets mapped to the known offset, collecting traces that span three times the workload's time window. To accelerate scanning, we retain only traces with plausible activity. Over 204 end-to-end attempts, 71.57% of scans correctly identify the target set within at most three passes. Successful scans average 1.24s, with a scanning rate of 1, 566 sets/s. Results include 4.9% false positives, which we mark in production if the key-extraction algorithm fails to extract the secret snippet from the subsequent recordings of the set.

5.4 Key Extraction

After classifying a trace as originating from the target set, the attacker reuses it to extract secret bit snippets. Prior work [48] applies supervised classification to label periodic boundary accesses in the target trace, using neighboring accesses within a fixed window. This method requires training data, struggles with noise shifts, and offers limited interpretability. We instead propose a structure-exploiting approach that leverages the constant-time nature of loop execution. For each reference point, we measure how far each access timestamp deviates from the nearest periodic position (period derived from the PSD) and select the reference that minimizes the sum of squared residuals (L2 loss). We then apply k-means clustering to these deviations to distinguish victim accesses from noise. Figure 4 illustrates the labeled victim accesses from a located target set's trace. We interpolate the remaining expected timestamps along the recovered subsequence and extract bits by checking whether an access occurs (within the time-resolution tolerance) at each expected position. Presence encodes a 1; absence encodes a 0. This unsupervised method provides key advantages over supervised learning. It resists noise without training data, generalizes across different target configurations (access periods and timing patterns), and improves interpretability. It also eliminates reliance on iteration-boundary accesses, since secret-dependent accesses themselves reveal a periodic structure. From the target trace, we extract contiguous bit snippets, separated by long inactivity gaps caused by desynchronization.

5.5 Results

We demonstrate end-to-end secret bit extraction in the AWS production environment. Our setup uses an `r6i.2xlarge` parent instance with a 12-way Intel Ice Lake-SP and 8 vCPUs, allocating two (equaling one full core) to the Nitro Enclave.

To highlight the exploitability of the leakage, we recover multiple bit snippets of a secret string processed in the enclave through single conditional accesses to a virtual address in constant time with a 4, 850-cycle period. We then greedily assemble and rank candidate strings using the most frequently observed k-grams. To evaluate the bit error rate (BER), we continually process a 128-bit secret nonce for 25% of the enclave's runtime. Once we identify the target set, we collect traces covering three times the workload's duration. In practice, we extract bits from the same trace used to locate the target set, assuming ephemeral workloads. Across 19, 996 target traces, we recover a median of 128 secret nonce bits (121 bits on average) with a median BER of 5.47% per trace (9.72% on average) and a standard deviation of 10.94%. Desynchronization produces outliers clustered near the maximal BER of 0.5. We evaluate how same-line activity degrades key recovery by injecting noise i.e. pseudo-randomly accessing the victim's cache line at varying rates. For each noise level, we measure mean BER across 100 traces and 20 repeats (Figure 5). We find that while frequency-domain detection rates remain high, extraction quality drops substantially with increased noise—yet stays practical at EC2-like levels.

We also recoded the attack and notified AWS. AWS has confirmed this attack and pointed out the dedicated-hosts services to handle any kind of side-channel attacks, but it comes with a higher cost.

6 Discussion

6.1 Other Enclave Technologies

Other enclave technologies such as Intel SGX, Intel TDX, AMD SEV-SNP, and Huawei Qingtian Enclaves also share the LLC with the physical host or co-located VMs, making them potentially vulnerable to LLC side-channel attacks. In our study, we examined Intel SGX and Huawei Qingtian Enclaves and confirmed their susceptibility to these attacks.

With Intel SGX for instance, we reproduce the evaluation experiment locally, successfully establishing the covert-channel between the SGX Enclave and the quiescent local host. Our setup involves a 12-way Intel Ice Lake (38-core Intel Xeon Platinum 8368 CPU at 2.4 GHz, 57 MiB LLC) and yields 1,216 possible eviction sets for the targeted address offset. Due to the absence of noise, we consistently identify the target set in the first scan pass. Across 10,000 target traces, we extract a median value of 120 bits (112 bits on average) only with a 3.73% median BER (5.24% on average) and a standard deviation of 6.48%. With less noise-induced accesses falsely labeled as stemming from the victim, the extraction algorithm isolates short snippets at the edges of the captured time window with precision, yielding better BERs. Desynchronization exhibits through shorter extracted snippets in that case. As future work, we plan to extend our investigation to additional enclave technologies such as Intel TDX or AMD SEV-SNP.

6.2 Mitigation

Mitigation strategies encompass hardware-based approaches like partitioning with Intel's Cache Allocation Technology (CAT) as in CATalyst, which isolates sensitive data in dedicated LLC ways to block interference with a minimal 0.7% overhead on benchmarks, or fully associative designs like MIRAGE employing skewed hashing, extra invalid tags ($50-75\%$), and cuckoo relocation to eliminate set conflicts at 2% slowdown and $18-21\%$ power increase while resisting discovery [21][36]; software defenses include constant-time and data-oblivious programming to eliminate secret-dependent timing and memory-access variations respectively, manual cache flushing on context switches, cache-oblivious algorithms [8], and detection via performance counters for anomalous misses, though stealthy attacks like Demote+Demote evade monitoring by avoiding extra cache events [34]. Hybrid methods, such as non-inclusive LLCs (which limit cross-core evictions, though the Snoop Filter can still be exploited), dynamic resizing [40], noise injection via shuffling, and enclave-based concealment (e.g., MAGE), offer layered protection. However, challenges remain in scalability, energy overhead, and coverage against adaptive adversaries. Recent research advocates principled, low-overhead defenses that target root causes like set conflicts rather than just obscuring them [42, 43].

To mitigate potential security risks, we recommend that enclaves should not share the LLC with any other EC2 virtual machines, including their parent instance. This isolation can be achieved either by allocating a dedicated NUMA node; where the enclave runs on an entire node while the parent EC2 instance uses another, a method that provides strong separation but is resource-intensive and costly or by leveraging hardware-based cache partitioning, which enforces strict LLC separation between the enclave VM and other VMs and helps defend against cache-based side-channel attacks.

7 Related Work

LLC Side-Channel Attacks. These attacks leverage the shared architecture of LLCs in modern multi-core processors [42]. While L1 and L2 caches are usually core-private, the LLC is shared among all cores, enabling attackers to eavesdrop on victim processes across different cores, VMs, or even enclaves by observing cache contention and access patterns. Foundational works include Flush+Reload [45], which uses shared libraries for precise monitoring, and Prime+Probe [20], which reveals information through cache set contention without shared memory, including in cloud settings. Later innovations like Flush+Flush [13] offer stealthier detection via execution timing, bypassing performance counters. As hardware defenses evolved, such as non-inclusive caches and randomized mappings, research adapted with techniques like Prime+Scope [31] for better temporal accuracy, Slice+Slice Baby [25] for millisecond-scale eviction set creation in transient scenarios, TikTag [17] for combining caches with speculation to undermine ARM Memory Tagging Extensions (MTE), and GhostRace [32] for exploiting synchronization flaws. Despite mitigations like Intel CAT partitioning, LLCs continue to leak via metadata and contention. ZenLeak [43] marks the first practical LLC side-channel attack on AMD's non-inclusive Zen processors, addressing hurdles in reverse-engineering cache addressing, building eviction sets, and evicting private cache lines across cores; it targets AMD Ryzen 9 5900X (Zen 3) with a Prime+Signal+Probe variant to exploit crypto flaws in tools like OpenSSLs AES T-tables.

In cloud contexts, these attacks have been proven feasible despite challenges like short execution windows and multi-tenant interference, with advanced eviction set construction techniques, such as L2-driven address filtering to reduce candidates by approximately 1/16 and binary search-based algorithms for $O(n \log n)$ efficiency, enabling full LLC set coverage in 2.4 minutes with 99.1% success rates on platforms like Google Cloud Run, far surpassing prior methods that took hours and had low yields [48], and demonstrated through end-to-end cross-tenant exploits recovering a median value of 81% of the ECDSA nonce bits in under 20 seconds with 3% error rates across numerous traces [48].

Side-Channel Attacks in the Cloud. Prime+Probe attacks [15, 16] have been executed on AWS EC2 to recover RSA keys, employing a reverse-engineered LLC slice hash function and huge pages for eviction set creation. However, the approach is prolonged, dependent on huge pages, and geared toward inclusive LLCs, making it ill-suited for current cloud infrastructures. Ristenpart et al. [35] analyzed virtual machine placements on AWS physical hosts and created methods for ensuring co-location. Flush+Reload [46] has been applied in cross-tenant assaults within Platform-as-a-Service (PaaS) clouds. More recent studies [48] prove that Prime+Probe attacks on LLCs are feasible amid noisy modern clouds, including Google Cloud Run on Intel Skylake-SP

processors, facilitating cross-tenant retrieval of secret bits from insecure ECDSA setups. Our research expands on this by facilitating LLC side-channel attacks on enclave technologies, such as AWS Nitro Enclaves, Huawei Qingtian Enclaves, and Intel SGX.

8 Conclusion

This paper reveals a critical side-channel vulnerability in AWS Nitro Enclaves stemming from the shared LLC with the parent EC2 VMs. The shared LLC enables adversaries to exploit cache-contention patterns to infer enclave computations without requiring direct memory access. We demonstrate practical cache-based attacks against real enclave workloads, report the vulnerability to AWS, and extend our analysis to Huawei Qingtian Enclave, notifying the vendor as well. Both providers have confirmed the issue. Finally, we outline mitigation strategies to counter these LLC-based threats.

9 Acknowledgments

AI tools assisted with wording and proofreading. We thank Prof. Yuval Yarom and the anonymous reviewers for their helpful feedback.

References

[1] 2025. AWS Nitro Enclaves. https://aws.amazon.com/ec2/nitro/nitro-enclaves/. Accessed: Nov, 2025.

[2] 2025. Confidential Compute Architecture (ARM-CCA). https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture. Accessed: Nov, 2025.

[3] 2025. Intel Trust Domain Extensions. https://software.intel.com/content/dam/develop/external/us/en/documents/tdx-whitepaper-v4.pdf. Accessed: Oct. 2025.

[4] 2025. The security design of the aws nitro system - Whitepaper. https://docs.aws.amazon.com/pdfs/whitepapers/latest/security-design-of-aws-nitro-system/security-design-of-aws-nitro-system.pdf. Accessed: Nov, 2025.

[5] Ittai Anati, Shay Gueron, P. Simon Johnson, and R. Vincent Scarlata. 2013. Innovative technology for CPU based attestation and sealing. In *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*.

[6] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software Grand Exposure: SGX Cache Attacks Are Practical. In *11th USENIX Workshop on Offensive Technologies (WOOT)*.

[7] Robert Buhren, Hans-Niklas Jacob, Thilo Krachenfels, and Jean-Pierre Seifert. 2021. One glitch to rule them all: Fault injection attacks against amd's secure encrypted virtualization. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2875–2889.

[8] T-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. 2018. Cache-oblivious and data-oblivious sorting and applications. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*.

[9] Shuwen Deng, Wenjie Xiong, and Jakub Szefer. 2019. Analysis of Secure Caches using a Three-Step Model for Timing-Based Attacks. Journal of Hardware and Systems Security volume 3.

[10] Johannes Götzfried, Moritz Eckert, Sebastian Schinzel, and Tilo Müller. 2017. Cache Attacks on Intel SGX. In *Proceedings of the 10th European Workshop on Systems Security (EuroSec)*.

[11] Franz Gregor, Wojciech Ozga, Sébastien Vaucher, Rafael Pires, Sergei Arnautov, André Martin, Valerio Schiavoni, Pascal Felber, Christof Fetzer, et al. 2020. Trust management as a service: Enabling trusted execution in the face of byzantine stakeholders. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*.

[12] Daniel Gruss, Julian Lettner, Felix Schuster, Olya Ohrimenko, Istvan Haller, and Manuel Costa. 2017. Strong and efficient cache side-channel protection using hardware transactional memory. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security 17)*.

[13] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. Flush+ flush: A fast and stealthy cache attack. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*.

[14] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. 2015. Cache template attacks: Automating attacks on inclusive {Last-Level} caches. In *24th USENIX Security Symposium (USENIX Security 15)*.

[15] Mehmet Sinan Inci and et al. 2016. Cache attacks enable bulk key recovery on the cloud. In *International Conference on Cryptographic Hardware and Embedded Systems*. Springer.

[16] Mehmet Sinan Inci, Berk Gulmezoglu, Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. 2015. Seriously, get off my cloud! Cross-VM RSA Key Recovery in a Public Cloud. *Cryptology ePrint Archive* (2015).

[17] Juhee Kim, Jinbum Park, Sihyeon Roh, Jaeyoung Chung, Youngjoo Lee, Taesoo Kim, and Byoungyoung Lee. 2025. TIKTAG: Breaking arm's memory tagging extension with speculative execution. In *2025 IEEE Symposium on Security and Privacy (SP)*.

[18] Do Le Quoc, Franz Gregor, Sergei Arnautov, Roland Kunkeland, Pramod Bhatotia, and Christof Fetzer. 2020. secureTF: A Secure TensorFlow Framework. In *Proceedings of the 21st International Middleware Conference (Middleware)*. ACM.

[19] Do Le Quoc, Franz Gregor, Jatinder Singh, and Christof Fetzer. 2019. Sgx-pyspark: Secure distributed data analytics. In *The World Wide Web Conference (WWW)*.

[20] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. 2015. Last-level cache side-channel attacks are practical. In *2015 IEEE symposium on security and privacy*.

[21] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. 2016. CATalyst: Defeating Last-Level Cache Side Channel Attacks in Cloud Computing. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

[22] Xiaoxuan Lou, Tianwei Zhang, Jun Jiang, and Yinqian Zhang. 2021. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. *ACM Computing Surveys (CSUR)* (2021).

[23] Frank McKeen, Ilya Alexandrovich, Alex Berenzon, Carlos V. Rozas, Hisham Shafi, Vedvyas Shanbhogue, and Uday R. Savagaonkar. 2013. Innovative instructions and software model for isolated execution. In *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*.

[24] Saeid Mofrad, Fengwei Zhang, Shiyong Lu, and Weidong Shi. 2018. A comparison study of Intel SGX and AMD memory encryption technology. In *Proceedings of the 7th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*.

[25] Bradley Morgan, Gal Horowitz, Sioli O'Connell, Stephan van Schaik, Chitchanok Chuengsatiansup, Daniel Genkin, Olaf Maennel, Paul Montague, Eyal Ronen, and Yuval Yarom. 2025. Slice+ Slice Baby: Generating Last-Level Cache Eviction Sets in the Blink of an Eye. In *2025 IEEE Symposium on Security and Privacy (SP)*.

[26] Alexander Nilsson, Pegah Nikbakht Bideh, and Joakim Brorsson. 2020. A survey of published attacks on Intel SGX. *arXiv preprint arXiv:2006.13598* (2020).

[27] Oleksii Oleksenko, Bohdan Trach, Robert Krahn, Mark Silberstein, and Christof Fetzer. 2018. Varys: Protecting SGX Enclaves from Practical Side-Channel Attacks. In *2018 USENIX Annual Technical Conference (USENIX ATC)*.

[28] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache attacks and countermeasures: the case of AES. In *Cryptographers' track at the RSA conference*.

[29] Wojciech Ozga, Christof Fetzer, et al. 2021. Perun: Confidential Multi-stakeholder Machine Learning Framework with Hardware Acceleration Support. In *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer.

[30] Wojciech Ozga, Do Le Quoc, and Christof Fetzer. 2021. TRIGLAV: Remote Attestation of the Virtual Machine's Runtime Integrity in Public Clouds. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*.

[31] Antoon Purnal, Furkan Turan, and Ingrid Verbauwhede. 2021. Prime+Scope: Overcoming the observer effect for high-precision cache contention attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*.

[32] Hany Ragab, Andrea Mambretti, Anil Kurmus, and Cristiano Giuffrida. 2024. {GhostRace}: Exploiting and mitigating speculative race conditions. In *33rd USENIX Security Symposium (USENIX Security 24)*.

[33] Ashay Rane, Calvin Lin, and Mohit Tiwari. 2015. Raccoon: Closing digital side-channels through obfuscated execution. In *Proceedings of the 24th USENIX Security Symposium (USENIX Security 15)*.

[34] Fabian Rauscher, Carina Fiedler, Andreas Kogler, and Daniel Gruss. 2025. A Systematic Evaluation of Novel and Existing Cache Side Channels. In *Network and Distributed System Security (NDSS) Symposium*.

[35] Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. 2009. Hey, you, get off of my cloud: exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM conference on Computer and communications security*.

[36] Gururaj Saileshwar and Moinuddin K. Qureshi. 2021. MIRAGE: Mitigating Conflict-Based Cache Attacks with a Practical Fully-Associative Cache. In *USENIX Security Symposium*.

[37] Michael Schwarz, Samuel Weiser, Daniel Gruss, Clémentine Maurice, and Stefan Mangard. 2017. Malware guard extension: Using SGX to conceal cache attacks. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*.

[38] AMD Sev-Snp. 2020. Strengthening VM isolation with integrity protection and more. *White Paper, January* (2020), 8.

[39] Jatinder Singh, Jennifer Cobbe, Do Le Quoc, and Zahra Tarkhani. 2021. Enclaves in the clouds: Legal considerations and broader implications. *Commun. ACM* (2021).

[40] Daniel Townley, Kerem Arıkan, Yu David Liu, Dmitry Ponomarev, and Oğuz Ergin. 2022. Composable Cachelets: Protecting Enclaves from Cache Side-Channel Attacks. In *31st USENIX Security Symposium (USENIX Security 22)*.

[41] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2020. SGAxe: How SGX fails in practice.

[42] Gaurav Verma, Ankit Pulkit, and Biresh Kumar. 2023. A Survey of Side-Channel Attacks in Context of Cache. *arXiv preprint arXiv:2312.11094* (2023). https://arxiv.org/abs/2312.11094

[43] Han Wang, Ming Tang, Quancheng Wang, Ke Xu, and Yinqian Zhang. 2025. ZenLeak: Practical Last-Level Cache Side-Channel Attacks on AMD CPUs. In *Design Automation Conference (DAC)*.

[44] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A. Gunter. 2017. Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*.

[45] Yuval Yarom and Katrina Falkner. 2014. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*. 719–732.

[46] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2014. Cross-tenant side-channel attacks in PaaS clouds. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*.

[47] Zirui Neil Zhao, Adam Morrison, Christopher W. Fletcher, and Josep Torrellas. 2024. Everywhere All at Once: Co-Location Attacks on Public Cloud FaaS. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*.

[48] Zirui Neil Zhao, Adam Morrison, Christopher W Fletcher, and Josep Torrellas. 2024. Last-level cache side-channel attacks are feasible in the modern public cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.